

censuschat — how it actually works

This walks through the seven mechanisms that decide how this system behaves. Each one is explained the same way: the mental model, a concrete example, the code that implements it, why it was built that way, and what goes wrong.

Every trace in this document was produced by running the real code, not written from memory.

0 **Orientation** — one question, end to end

1 **Where knowledge lives** — what goes in the prompt, what goes behind search

2 **The SQL gate** — the only thing standing between the model and the warehouse

3 **The tool loop** — three separate limits, and why they are separate

4 **The ambiguity backstop** — and the one-line bug that made it fake

5 **The guardrail** — why it deliberately fails open

6 **What an eval run measures** — and what it cannot

7 **The incident** — when good error handling hides total failure

— Appendix: the assignment's criteria, mapped to these parts

0 Orientation — one question, end to end

Before any individual mechanism makes sense, you need the sequence they sit in. Here is what actually happens when someone types *"What is the total population of Wyoming?"*

Browser —POST /api/chat {session_id, message}—> Caddy (TLS, basic auth)

FastAPI src/app.py

agent_turn src/agent.py

1. **degraded?** snapshot + Snowflake checked once at boot → no, continue
2. **guardrail** Haiku classifies the message (1.5s timeout)
verdict: on_topic → ALLOW
3. **model call** Sonnet + full session history + 3 tool defs
decides: resolve_geography("Wyoming")
4. **tool** local SQLite — no network
→ {geo_id: "56", ambiguous: false}
5. **model call** decides: search_census_variables("total population")
6. **tool** local SQLite FTS5 — no network
→ B01003e1, geo_levels: [all five]
7. **model call** writes SQL, calls run_census_sql(...)
8. **THE GATE** validate_sql() ← the trust boundary
parse → single statement → is a query → no banned
construct → tables allowlisted → LIMIT injected
→ approved, rewritten SQL returned
9. **Snowflake** executes; STATEMENT_TIMEOUT 25s → 581,348
10. **model call** writes the answer using only returned rows
11. **persist** both messages to SQLite, keyed by session_id
stream terminates with `done`

Three things in that sequence explain most of the design, and each gets its own part below.

Steps 4 and 6 never touch the network. Two of the three tools read a local SQLite snapshot built once at boot. Only step 8–9 reaches Snowflake.

Step 8 is the only place a query can reach the warehouse, and it defaults to refusing. Everything above it — the prompt, the classifier — can be talked past by a sufficiently creative input. This cannot, because it is code with no model in the loop.

Steps 3, 5, 7 and 10 are four separate model calls in one turn. That is why the system prompt is cached, and it is why "one question" can take 10 to 30 seconds.

WHAT TO REMEMBER FROM PART 0

A turn is a loop, not a pipeline. The model is re-invoked after every tool result, and it decides what to do next each time. The loop is bounded three different ways (part 3), and the only exit to the outside world is guarded by one function (part 2).

1 Where knowledge lives

```
src/agent.py SYSTEM_PROMPT · src/tools.py search_census_variables
```

MENTAL MODEL

Every data-grounded agent has to answer one question: how does the model find out what is in the database?

There are only two places to put that knowledge — in the prompt, or behind a tool call. The useful test for deciding is not "which facts are important." It is **is this set closed or open?**

A closed set is small and never changes, so it fits in a prompt and will not drift. An open set is large and unpredictable, so it belongs behind search.

CONCRETE EXAMPLE

This dataset has both kinds, which is what makes it a clean illustration.

Closed: every table is at census-block-group grain, and the 12-character block group key splits by position:

```
560210012001
56|021|001200|1
├──┬──┬──┬──┐
│ │ │ │   │ block group
│ │ │ └───┘ tract
│ │ └──────┘ county  (021 = Laramie)
│ └──────────┘ state   (56 = Wyoming)
└────────────────┘
```

so: state total = SUM over rows WHERE SUBSTR(key,1,2) = '56'

```
so: state total = SUM over rows WHERE SUBSTR(key,1,2) = '56'
```

That is about five facts and they will never change. They go in the prompt.

Open: 8,164 field codes, 3,782 of them indexed estimates, across 298 ACS tables. Nobody can predict which one a user needs.

So when the model needs "total population," it does not recall `B01003e1` from its prompt. It calls a tool, and the tool searches.

HOW THE CODE WORKS

The system prompt carries the key decomposition and one worked aggregation pattern. It contains **zero variable IDs**. The only code-looking thing in it is `B19xxx`, which demonstrates

how a table number maps to a physical table — a pattern, not a field.

Search is SQLite FTS5 with BM25 ranking, over a snapshot built once at boot. The one line that matters is how the query is turned into a match expression:

```
# src/tools.py – user types "number of households"
_fts_match_query("number of households")
→ '"number" "of" "households"'      # FTS5: all three required

_fts_match_query("number of households", operator="OR")
→ '"number" OR "of" OR "households"' # any one is enough
```

WHY IT WAS DESIGNED THIS WAY

Requiring all tokens gives high precision, and a probe measured it at 7/7 recall on in-corpus queries. That is why it was chosen.

Then it failed in production, and the failure is instructive. Real agent queries carry framing words the Census catalog never uses — "number of," "distribution of," guessed table IDs. Under all-tokens-required, one such word drops the result count to zero.

Observed live: seven searches in a row returning nothing, while the variable the agent needed sat in the snapshot the entire time.

The fix only adds behavior. Try all-tokens. If and only if that returns nothing, retry the identical query with any-token.

THE PART WORTH COPYING

The fallback was measured before it was trusted. On the real snapshot, every query that already worked returned *the identical top 3, in the same order*, under the looser rule.

So the relaxation is provably inert on the cases that were already fine. When you loosen a constraint, prove that — do not assume it, and do not assume the opposite either.

COMMON MISTAKES AND TRADEOFFS

The common mistake: curate twenty "important" variables into the system prompt. This demos beautifully, because the demo questions came from the same list. It breaks on the twenty-first question, and it breaks quietly — the model has been shown a list, so it picks the nearest item instead of saying the concept is missing.

The subtler mistake, and this project made it in writing first: seeing empty searches and concluding "the model is making too many exploratory calls," then raising the tool-call limit. The model's queries were fine. Search was returning nothing. Raising the limit buys more empty searches and buries the real defect under a higher ceiling.

The accepted tradeoff: any-token ranking is genuinely worse for some queries. "Number of households" can surface *Median Number Of Rooms*, because "number" is rare and BM25 rewards rarity. That is real. It is still far better than an empty result, because a mediocre ranked list is something the model can refine against, while nothing gives it no signal and costs a whole round.

WHAT TO REMEMBER FROM PART 1

Closed and small → prompt. Open and large → retrieval. Decide with that question, not with "what seems important."

2 The SQL gate

src/sqlgate.py · 175 of the project's 345 tests

MENTAL MODEL

The model writes SQL. That SQL runs against a real warehouse with real credentials. Something has to decide whether each statement is allowed to run.

The important idea is the difference between a **soft layer** and a **boundary**:

LAYER	KIND	WHAT HAPPENS WHEN IT FAILS
System prompt instructions	Soft	The model may ignore them
Guardrail classifier	Soft	Fails open — allows the turn
<code>validate_sql</code>	Hard	The query never reaches Snowflake

Only the third one is a boundary, because it is the only one with no model in the decision path. It is a pure function — no I/O, no network, no clock — which is exactly what makes it exhaustively testable.

CONCRETE EXAMPLE

Here is the real function's actual output on seven inputs. Watch the last two especially: the gate does not just approve or reject, it *rewrites*.

INPUT	RESULT
SELECT SUM("B01003e1") AS pop FROM ..."2020_CBG_B01" WHERE SUBSTR(key,1,2) = '56'	approved , rewritten as ... WHERE SUBSTRING(key, 1, 2) = '56' LIMIT 200
SELECT * FROM ..."2020_CBG_B01"	rejected — banned_construct "star projection is not permitted; name the columns"
SELECT key, SYSTEM\$CANCEL_ALL_QUERIES() FROM ..."2020_CBG_B01"	rejected — banned_construct "unrecognised function(s): SYSTEM\$CANCEL_ALL_QUERIES"
SELECT SUM("B01003e1") FROM ..."2019_CBG_B01"	rejected — table_not_allowed

INPUT	RESULT
SELECT 1	rejected — "query does not read any allowed table"
SELECT key FROM ... LIMIT 100000	approved , clamped to LIMIT 200
SELECT key FROM ... LIMIT 5	approved , left at LIMIT 5

LOOK AT THESE THREE THINGS

1. **SUBSTR** became **SUBSTRING**. The gate parses the SQL into a syntax tree and renders it back out. What runs is not the model's text — it is the gate's re-rendering of a tree it fully understood. If it cannot understand a construct, it cannot render it, and the query dies.
2. **LIMIT 5** survived but **LIMIT 100000** was **clamped**. The row cap is a ceiling, not a default. The agent may have a good reason to want five rows, and rewriting that would be the gate editing intent rather than enforcing a bound.
3. **SELECT 1** was **rejected**. Not "nothing to check" — rejected. That is the shape every escape takes that reads data without naming a table.

HOW THE CODE WORKS

Seven checks run in order: it parses; there is exactly one statement; it is not an opaque command; it is a query; no banned construct appears anywhere in the tree; at least one table is named and all named tables are allowlisted; a row limit is applied.

Of the ~430 lines, these three decisions carry almost all of the security:

```
# 1. Permitted statement types are LISTED. Everything else is
#   denied by omission – including node types that don't exist yet.
_QUERY_TYPES = (exp.Select, exp.SetOperation, exp.Subquery)

# 2. "The parser has no specific node for this" is used as a
#   stand-in for "this is not ordinary SQL."
names = {str(n.this).upper() for n in statement.find_all(exp.Anonymous)}

# 3. Zero tables is a rejection, not a no-op.
if not referenced:
    return "query does not read any allowed table"
```

Decision 2 is the one worth understanding. sqlglot models ordinary SQL functions as specific typed nodes, and falls back to a generic `Anonymous` node for anything it does not recognize.

So "landed in `Anonymous`" is a usable proxy for "outside ordinary SQL" — which catches user-defined and external functions whose names could be anything.

WHY IT WAS DESIGNED THIS WAY

The alternative is a blacklist: ban `DROP`, `DELETE`, semicolons. The argument against it is evidence, not theory.

Three separate bypasses got past this gate's *first* implementation — written by someone specifically trying to make it airtight — and were found only by attacking the real function. Two are worth seeing, because they are not obvious:

```
-- BYPASS A: the allowlisted FROM clause launders nothing.
-- Nothing being checked was looking at the SELECT list.
SELECT key, SYSTEM$CANCEL_ALL_QUERIES() FROM ...allowed...

-- BYPASS B: a decoy CTE named after a banned table.
-- The bare reference in branch 1 is NOT in scope of the CTE
-- declared inside branch 2 – so Snowflake resolves it against
-- the REAL 2019 table.
SELECT key FROM ...allowed...
WHERE key IN (SELECT key FROM "2019_CBG_B01")  ← real table
UNION ALL
SELECT x FROM (
  WITH "2019_CBG_B01" AS (SELECT 1 AS x)      ← decoy, other scope
  SELECT x FROM "2019_CBG_B01"
) AS decoy

Both now rejected. Bypass B reports:
table(s) not in the allowlist: 2019_CBG_B01
```

Bypass B is why CTE names are resolved *by scope* — walking up the tree from each reference — rather than collected into one flat set. A flat set excuses a bare table name anywhere in the query as soon as any branch happens to define that name.

There is a second version of the same bug: a CTE declared *after* the reference that uses it. A CTE cannot see one declared later, so Snowflake again falls through to the real table. Visibility therefore follows declaration order.

Why the 2019 tables matter enough to defend this hard: block groups were redrawn for the 2020 census and the five-year windows overlap in four of five years, so comparing the vintages is statistically invalid. Leaving 2019 off the allowlist makes the invalid query *impossible* rather than discouraged.

COMMON MISTAKES AND TRADEOFFS

The common mistake: a regex denylist. If a purpose-built parser-based gate had three holes, a regex has thirty. The quieter version: checking only the first `FROM` clause, which passes every bypass above.

The measured tradeoff: refusing every function the parser cannot model is aggressive. So the cost was measured rather than assumed — of 48 functions a census answer plausibly needs, 47 pass. One is lost.

The limit of this design, stated honestly: at production scale the gate should not be the only boundary. The real control is a Snowflake role with `SELECT` on exactly those 31 objects and nothing else, so a gate bug is contained by the database rather than by a parser. The gate stays as defense in depth and as the thing that produces good error messages.

WHAT TO REMEMBER FROM PART 2

- List what is allowed, not what is banned. New things then land on the deny side by default.
- Parse, do not pattern-match. Re-rendering from a syntax tree means anything unparseable cannot survive.
- "Zero of the thing I check for" is usually a rejection, not a pass.

3 The tool loop and its three limits

src/agent.py · CLAUDE.md rules 9 and 11

MENTAL MODEL

After each tool result the model is called again and decides what to do next. Nothing in that arrangement makes it stop. Three different things can go wrong, and they need three different limits — conflating them is the mistake.

LIMIT	VALUE	BOUNDS WHAT
Recovery budget	2	Retries <i>after a failure</i> — bad SQL or zero rows
Loop iterations	8	Total rounds <i>even when everything succeeds</i>
Wall-clock watchdog	50s	Elapsed time, regardless of the other two

CONCRETE EXAMPLE

A question that is genuinely multi-step — "what's the average household income in Texas?" — needs a numerator and a denominator, so it needs several searches before it can write any SQL. Here is what actually happens:

```
round 1  search "average household income"    ok  recovery: 0/2
round 2  search "aggregate household income"   ok  recovery: 0/2
round 3  resolve_geography "Texas"             ok  recovery: 0/2
round 4  search "number of households"         ok  recovery: 0/2
...
round 8  loop cap reached
        "[Stopped after reaching this turn's tool-call limit.]"
        elapsed 30.7s — the watchdog never fired
        recovery budget never spent — nothing failed
```

This is the point of keeping the limits separate. Nothing failed, so the recovery budget is untouched. It finished well inside 50 seconds, so the watchdog is irrelevant. It ran out of *rounds*. If those three had been one counter, the diagnosis would have been impossible.

HOW THE CODE WORKS

The counting rule is the part that matters:

```
# Only run_census_sql spends the recovery budget – and BOTH
# a hard failure and a zero-row result count as spending it.
if block.name == "run_census_sql":
    if is_error:
        recovery_attempts += 1
    elif result_payload.get("row_count") == 0:
        recovery_attempts += 1
    else:
        collected_query_results.append(result_payload)
```

When any limit trips, the message the user sees is **generated by code, not by the model**. Recovery exhaustion lists every attempt made. The watchdog formats rows already returned.

WHY IT WAS DESIGNED THIS WAY

All three started as prompt instructions and were moved into code after being watched to fail.

The recovery budget originally counted only queries the *gate* rejected. That left real Snowflake execution errors uncounted — so a model guessing bad SQL could retry without limit, at full warehouse cost. That is precisely the exposure the rule exists to prevent.

THE PRINCIPLE UNDERNEATH

A promise a language model can decline is not a promise.

"At most 2 retries" is a commitment about money and latency made to whoever operates this system. If the only thing enforcing it is a sentence in a prompt, it is advisory.

The test for where a rule belongs: does violating it cost money, leak data, or produce a silently wrong number? If yes, it goes in code.

COMMON MISTAKES AND TRADEOFFS

The mistake in the watchdog's design — and it is deliberate, not accidental: when time runs out, do *not* ask the model to summarize what it found. That spends more of the budget already gone, and risks the model narrating a number no query actually returned. The partial message formats the real rows directly.

The honest weakness: the watchdog checks the clock *between* tool rounds. It cannot abort a model call already in flight. So it is a soft bound at 50 seconds against a 60-second requirement, and the 10-second gap is the margin. The right fix is per-call timeouts and real cancellation, not a bigger budget.

WHAT TO REMEMBER FROM PART 3

Retries, rounds, and elapsed time are three different failure modes. One counter for all three tells you nothing when it trips.

4 The ambiguity backstop

src/agent.py unresolved_ambiguous_geo · D-014

MENTAL MODEL

Thirty states have a Washington County. When a lookup is ambiguous, the correct behavior is to ask — never to quietly pick the biggest or the first.

The interesting question is *where* that rule lives. Asking a question is model behavior, so the instinct is to put it in the prompt. But the failure mode here is a confidently stated number for the wrong county, with no error and nothing in the answer showing a choice was ever made. That is not a behavior problem, it is a correctness boundary.

CONCRETE EXAMPLE

This is the bug a code review found, and it is worth tracing because the trigger is completely ordinary — any comparison question naming an ambiguous county alongside a real one.

```
"Compare Washington County with Travis County, Texas"

--- the ORIGINAL implementation: one variable ---

resolve_geography("Washington County")
  → ambiguous, 30 candidates
  pending_ambiguous = {30 candidates}      ← guard armed

resolve_geography("Travis County, Texas")
  → unambiguous, 1 candidate
  pending_ambiguous = None                  ← guard silently DISARMED

run_census_sql(...)
  → allowed through. Model picks a Washington County.
  No error. No indication a choice was made.

--- the FIX: a list, only ever appended to ---

  unresolved_ambiguous_geo = [{30 candidates}]  ← stays armed
run_census_sql(...)
  → BLOCKED. Turn ends with a clarifying question
  naming every candidate. Snowflake never touched.
```

HOW THE CODE WORKS

The fix is a data-structure change, not more logic. The variable became a list that is appended to on every ambiguous result and never cleared for the rest of the turn:

```
unresolved_ambiguous_geo: list[dict] = []

# armed by any ambiguous lookup
elif block.name == "resolve_geography" and result_payload.get("ambiguous"):
    unresolved_ambiguous_geo.append(result_payload)

# checked before the only tool that reaches Snowflake
if block.name == "run_census_sql" and unresolved_ambiguous_geo:
    ambiguity_blocked = True
    break
```

The clarifying question is then built by code from every unresolved entry — not just the most recent one, since a turn can raise two ambiguities.

WHY IT WAS DESIGNED THIS WAY

It is enforced twice on purpose: as a prompt instruction, so the model normally does the right thing gracefully, and as a code block, so that when it does not, the consequence is a clarifying question rather than a wrong number.

The prompt handles the common case well. The code handles the case the prompt misses. Neither alone is sufficient.

COMMON MISTAKES AND TRADEOFFS

The mistake this is: a guard whose state can be wiped by unrelated success. It is not a guard — it is a guard-shaped object that passes its own happy-path test. Every test written for it passed, because every test exercised one ambiguous lookup in isolation.

The transferable version: for a safety property, prefer state that only accumulates. If clearing it is possible, something unrelated will eventually clear it.

WHAT TO REMEMBER FROM PART 4

When a guard tracks "is there an unresolved problem," make it append-only. A last-write-wins variable will be reset by something that has nothing to do with the problem.

5 The guardrail, and why it fails open

src/guardrail.py · D-019

MENTAL MODEL

There is a fast classifier (Haiku, 1.5-second timeout) in front of the agent loop. Its job is *speed*, not safety: reject off-topic and injection input in about 1.3 seconds with zero tool calls, so obvious junk never costs a full agent turn.

Because safety is not its job, it is built to be bypassable without consequence. Any error or timeout returns ALLOW.

CONCRETE EXAMPLE

Two questions that look similar and route completely differently:

```
"What's the weather in San Francisco?"  
guardrail → off_topic → canned refusal, 0 tool calls, ~1.3s  
  
"How many people will live in Texas in 2050?"  
guardrail → on_topic (it IS a census-shaped question)  
agent      → declines itself: the ACS measures the past  
              0 tool calls, ~4s
```

Both end in a refusal with no tool calls, but by different mechanisms. The eval scorer records *which* one fired rather than collapsing them, because the distinction is real.

HOW THE CODE WORKS

The whole fail-open contract is these lines:

```
try:  
    raw = _call_classifier_model(message, recent_turns)  
    ...  
except Exception:  
    return GuardrailVerdict(  
        action=GuardrailAction.ALLOW,          # ← fails OPEN  
        reason="classifier_unavailable",  
    )
```

There is also an explicit set of allow-verdicts, so a label that appears in neither the refuse set nor the allow set is reported as `unrecognized_verdict` rather than silently passing. A routing gap becomes visible instead of quiet.

WHY IT WAS DESIGNED THIS WAY

Failing closed on classifier errors feels safer and is not. It trades a common real failure — the classifier is down, every legitimate question is blocked, the product is offline — against a failure the classifier does not actually prevent. The SQL gate is what stops dangerous queries, and the gate does not care whether the classifier is up.

The over-refusal bug is the better lesson. "What's the population of Atlantis?" was being refused as off-topic, instead of reaching geography lookup and honestly reporting no match.

The cause was a layer-responsibility error: the classifier was being asked, in effect, to decide whether a place *exists*. It has no data access. It cannot know. The fix adds a category for exactly this shape:

```
unknown_subject: a demographic question in every respect except
that its subject may not exist in the data. Classify on the
SHAPE of the question; do not try to decide whether the data
exists. The assistant's tools will look and report honestly.
```

COMMON MISTAKES AND TRADEOFFS

The injection detail worth stealing: the classifier's own explanation text is never shown to the user and never persisted. Refusals render from a fixed message table keyed by category. If model-generated text about an adversarial input reached the client, that would be a channel for an attacker to get chosen text rendered.

The tradeoff, stated honestly: failing open is only defensible while something else bounds cost. Today nothing does — there is no rate limiting and no per-session spend cap, so a classifier outage plus a scripted client is an unbounded bill behind one password. At scale the classifier stays fail-open and rate limiting becomes the thing that fails closed.

WHAT TO REMEMBER FROM PART 5

Decide what each layer is *for*, then let it fail in the direction that matches. A fast filter should fail open; a correctness boundary should fail closed. Asking a layer a question it structurally cannot answer — like whether a place exists — produces exactly this class of bug.

6 What an eval run measures

evals/ · D-022

MENTAL MODEL

Unit tests are deterministic: same input, same result, so one run is proof. An eval against a live model is not. The same scenario can pass and then fail with nothing changed.

That means a single run is a *sample*, and a pass rate computed from one sample per row is not a measurement.

CONCRETE EXAMPLE

The current set is 14 examples. One of them, `PM-08`, passes roughly two runs in three:

```
run 1  PM-08 pass    "13/14 - 93%"    looks fixed
run 2  PM-08 fail    "12/14 - 86%"    looks like a regression
run 3  PM-08 pass    "13/14 - 93%"    looks fixed again
```

Nothing changed between those runs. No commit. No config.

```
--repeat 3 collapses them into one column:
    PM-08 2/3 ← rendered as its OWN state,
               not folded into pass or fail
```

HOW THE CODE WORKS

`--repeat N` runs the whole set N times and writes N result files. Files are keyed by `git_sha`, so runs of the same commit collapse into a single column in the history grid, showing how many passed.

The grid is per-scenario rather than per-run, which fixes a second problem. The first two recorded runs are 100% and the third is 93% — that looks like a regression and is not. The set grew from 11 examples to 14 and one addition is deliberately red. An aggregate rate is only comparable across runs whose denominator did not change.

WHY IT WAS DESIGNED THIS WAY

A red row is kept deliberately. `PM-08` was deleted from the set once and restored, because a run with the known failure removed is a worse artifact than a run that reads 13/14.

Its triage note records something most write-ups omit: the bottleneck **moved**. Search was the original cause and was fixed; the limiting factor is now the 8-round cap; raising that trades against the 50-second watchdog. "Fixed the first cause, still failing, here is the new limiting factor and what relieving it would cost" is far more useful than either a green row or an unexplained red one.

COMMON MISTAKES AND TRADEOFFS

The mistake: on seeing a flaky scenario, re-run until it passes and commit that file. Nobody experiences that as falsification at the time, and it is exactly how a suite stops measuring anything.

Where this suite under-verifies, named rather than hidden: one check asserts only that an answer contains the word "median" and does not error. What the design actually wants — an explanation of why medians cannot be aggregated — needs an LLM judge that was cut. The live answer did explain it correctly. The check would not have caught it if it hadn't.

The method for fixing that: take a known-good answer, break it deliberately, and confirm the check goes red. A check that has never failed has not been tested.

WHAT TO REMEMBER FROM PART 6

Any metric over a stochastic system needs a repeat count before it needs a threshold. And the failure is asymmetric — you stop re-running once it is green, so variance only ever gets recorded as progress.

7 The incident

docs/reflection.md §4

WHAT HAPPENED

280 tests green. Every layer verified against live Snowflake as it was built. Real queries had returned real grounded numbers.

Then a browser was opened for the first time and two questions were typed. Both failed.

Every single query in the system was failing.

SafeGraph created the ACS columns case-sensitively. Snowflake folds unquoted names to uppercase. And the system prompt's own worked example showed an unquoted column reference — teaching the model the one pattern guaranteed to fail against this database.

WHY THE TEST SUITE STRUCTURALLY COULD NOT CATCH IT

Three reasons, and they compound:

- **The gate is a safety boundary, not a correctness boundary.** It passed the query, correctly — single statement, allowlisted table, limit applied. No gate test can know whether a column name resolves against a real catalog, and none should.
- **The fake Snowflake connection encoded the same assumption the code did.** A mock written by the author of the code cannot disagree with the author of the code.
- **The defect lived in a prompt; its consequence lived in Snowflake's name resolution.** There is no artifact in between for a unit test to hold.

THE PART THAT GENERALIZES

The bug is not the valuable finding. This is:

A GOOD RECOVERY PATH RAISES THE COST OF NOTICING WHAT IT RECOVERS FROM

The defect had surfaced **twice before**, and both times it was logged as a success. The model hit the error, retried with quotes, got the right answer — recorded as the agent gracefully handling a Snowflake quirk.

Both log entries were true. Both described a systematic prompt defect as ordinary resilience.

It became visible only when two questions in a row failed to self-correct within budget — which happened immediately after the retry budget was correctly *tightened*. Better error handling did not cause the bug; it postponed the discovery of it.

The actionable version: **instrument retries as a first-class metric with a threshold**. "The agent self-corrected" is a signal, not a non-event. Every retry that succeeds is a failure nobody investigates.

WHAT IT CHANGED

The system prompt's correctness rules are now ordered by *what a violation costs*, and rule 1 is the quoting rule — stated with the mechanism attached, so the model knows *why* unquoted names fail rather than just that they do. A model given the reason generalizes to cases the rule never listed.

WHAT TO REMEMBER FROM PART 7

Mocked tests verify your code. Only a real end-to-end call verifies your system. They fail differently, and here only one of them was capable of noticing that the product did not work at all.

Appendix — the assignment's criteria, mapped

CRITERION	ANSWERED IN
LLM / AI Engineering — genuine understanding of data-grounded agents; defensible choices	Parts 1, 2, 3 — the closed/open split, the gate, the loop
Production Quality — would you ship it; behavior under ambiguous, incomplete, adversarial input	Parts 2, 4, 5 — the gate, the ambiguity backstop, the two guardrail directions
Judgment Under Constraints — where the time went, what was cut	Part 6, plus the cuts below
Reflection & Self-Awareness — honest assessment	Part 7, and the named weaknesses in 3, 5, 6
Comprehensive Mapping — do not limit yourself to a subset	Part 1 — all 3,782 estimate fields indexed; two classes excluded from search only, each for a stated reason
Context Awareness — see the schema without hard-coding it	Part 1 — zero variable IDs in any prompt
Deployment "Truth" — local doesn't count	Live behind Caddy with TLS and basic auth; the app binds to loopback, so auth cannot be skipped by hitting the port
Graceful Degradation — explain why, never crash	Part 0 step 1 — a boot with no snapshot or no Snowflake still serves and says so
Stateful Continuity — remember previous messages	Part 0 step 11 — full history replayed from SQLite; no summarization, so context cannot drift from what was said
Operational Guardrails — a dedicated validation layer	Parts 2 and 5
Latency Threshold — over 60s is a failure	Part 3 — streaming throughout, 50s watchdog, 25s query timeout, slowest scenario 30.7s
Safety First — correct and slow over fast and wrong	Part 5 fast-fail at ~1.3s; grounding rule means zero rows is "not found," never a number

THE FOUR CUTS, AND WHY EACH

CUT	SHIPPED INSTEAD	REASONING
Langfuse tracing	In-app trace tab: one trace per turn, spans per guardrail / tool / model call, real token counts	The span design took the judgment; the SDK and hosted project were the expensive half and buy nothing visible in a single-instance demo. Swapping the local buffer for an exporter is a change at one seam
Decennial tables	Nothing — one eval category has no coverage	This was the genuine second data source for "two defensible answers." Cost stated, not papered over
The LLM-judge grounding check	Six deterministic check types	Named as the biggest hole: the core grounding rule has no automated enforcement anywhere
18 of 30 designed scenarios	14 that all actually run	Designing them was free — it predated the code. Building the runner cost the time

Every trace, rejection message and figure in this document was produced by running the code in this repository.

Sources: `src/sqlgate.py`, `src/agent.py`, `src/guardrail.py`, `src/tools.py`, `evals/results/latest.json`, `docs/decisions.md` (D-001 ... D-022), `docs/reflection.md`. Current figures: 345 tests, 175 on the SQL gate; 14 eval examples, all of which have run, 13 passing.